SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100
CO2 Statement:	*Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.*		
Student Name:	DHIVYADHARSAN M.S	**Reg. No.:**	192511465
Section / Batch:	2025	**Date:**	20/07/2026

Code of Conduct

I __DHIVYADHARSAN M.S_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: DHIVYADHARSAN M.S_

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
	GRAND TOTAL:		**100 Marks**

Annexure A – Analytical Problem Solving

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO2 - AT1

Analytical Problem Solving

Name: M.S. Dhivyadharsan
Reg No: 192511465

Course Code: CSA0305
Course Name: Data structure
Date: 3/8/26

Q)1. Identify if the given expressions are balanced or not i) [9 * (K+8 - {6+e} + 77/100
ii) (((a*b)+[u/2] - d).

Ans: To identify whether the expression are balanced or not.

i) [9 * (K+8 - {6+e} + 77/100

Step - by step analysis using a stack:
→ Push [onto stack.
→ Push (onto stack.
→ Encounter) → POP (.
→ Push { onto stack.
→ Encounter } → POP {.
→ End of expression reached.
stack still contains [which has no matching].

Conclusion: The expression is Not balanced because one opening square bracket remains unmatched.

ii) (((a*b)+[u/2] - d).

step by step analysis using a stack:
→ Push (, (, (onto stack.
→ Encounter) → Pop one (
→ Push [.
→ Encounter] → Pop [.

→ End of expression reached.
Two opening parentheses (remain in the stack)
Conclusion: The expression is not balanced because two opening parentheses are unmatched.

Q)2. Given a stack with repeated elements, design an approach to compress it (Value & frequency) while maintaining stack operations. Apply stack ADT modifications. Analyze trade-off between time and space.

Ans: → To reduce memory usage in a stack with repeated elements, store each element as a (value, frequency) pair instead of storing duplicate elements multiple times.

Example:

Normal stack:

5, 5, 5, 3, 3, 2

Compressed stack:

(5, 3), (3, 2), (2, 1)

Operations:

→ Push(x): If the top element is the same as x, increase its frequency; otherwise push a new pair.

→ Pop(): Decrease the frequency if it is greater than

1; otherwise remove the pair.

→ Peek(): Returns the value of the top pair.

Complexity:

→ Push = O(1)

→ POP = O(1)

→ Peek = O(1)

Advantages:

→ Reduces memory consumption

→ Maintains efficient stack operations

Disadvantages:

→ Requires an additional frequency field.

Conclusion:

→ Using (value, frequency) pairs compresses the stack and saves space while keeping all stack operations efficient.